Task 5: Automated Testing — Unit, Integration, and End-to-End

Phase 2, Task 5 deliverable. Author: Ahmed. Date: 2026-11-12. Branch: `feature/automated-tests-ci`. Pull request description (with the "break it on purpose" demonstration): docs/PR_TASK5.md.

```
npm install
npx playwright install chromium   # first time only
npm test                          # Vitest: unit + integration (Node and jsdom + MSW)
npm run test:coverage             # the same with coverage; fails below 80% of lines in
state/ or utils/
npm run e2e                       # Playwright Test: the two end-to-end tests (Python 3.9+
unless VITE_API_BASE is set)
npm run e2e:prove                 # proves each e2e test fails when its feature is broken
```

1. What Was Built

Requirement	Where	Count
Unit tests (Vitest) for the pure cart functions, totals, shipping rule, currency formatting, query-string builder, price-string parser, storage utility (corrupted / missing data)	`tests/unit/edge-cases.test.js` (new) plus the existing `cart`, `query-string`, `saved-recent-storage`, `techsphere`, `validate` … files	174 unit tests (32 new)
Integration tests in jsdom rendering the catalog and cart modules against MSW handlers for 200, empty, 401, 429, 500	`tests/dom/catalog-ui.test.js`, `tests/dom/cart-ui.test.js`, handlers in `tests/support/api-handlers.js`	13 UI tests (+ 4 for `utils/dom.js`)
Two Playwright end-to-end tests	`tests/e2e/specs/catalog-cart.spec.js`, `tests/e2e/specs/account.spec.js`, `playwright.config.mjs`	2
≥ 80% line coverage on `state/` and `utils/`	`vitest.config.js` thresholds (per folder); `scripts/coverage-summary.mjs`	99.87% / 95.22% (see §5)
GitHub Actions: lint, unit + integration, e2e on every pull request	`.github/workflows/ci.yml` (3 jobs)	—
Each e2e test fails when its feature is broken	`scripts/prove-e2e.mjs` (`npm run e2e:prove`)	4 of 4 bugs caught

2. Unit Tests: Edge Cases, Not Only the Happy Path

Edge case	Examples tested	Expected result
Zero / negative quantity	`0`, `-1`, `-100`, `0.4`, `"0"`	corrected to 1 with "The smallest quantity is 1."; adding 0 leaves the cart unchanged; updating a line to 0 removes it
Not a number	`""`, `"abc"`, `NaN`, `Infinity`, `null`, `undefined`	"Enter a whole number of items." (never silently 0)

Edge case	Examples tested	Expected result
Above stock / above limit	stock 3 asked 4; 11 when the limit is 10; stock 0; five repeated adds; a stored line of 99	reduced to the stock or the limit with the reason; out of stock refused; repeated adds stop at the stock; stored data re-clamped
Floating-point totals	`0.10 + 0.20`; `3 × 1.10 + 0.10 + 0.20` (float gives 3.6000000000000005); `10 × 33.33`; $99.99 / $100.00	exact integer cents: 30, $3.60 + $7.99 = $11.59, $333.30, threshold exact
Price-string parser	`"19.99"`, `"1.005"` (float ×100 = 100.4999…), `"EGP 33,999"`, `"1,299.50"`, `0.29`	1999, 101, 3399900, 129950, 29; `""`, `"free"`, `null` → NaN, never 0
Currency formatting	0, 5 cents, $1,999.90, EGP, `de-DE` euros	`$0.00`, `$0.05`, `$1,999.90`, `EGP 33,999.00`, `1.299,50 €`
Query-string builder	null/empty values, `0`, `a&b=c #1`, `?page=-4&limit=9999&sort=evil&foo=bar`	empties dropped, 0 kept, characters encoded, invalid values replaced by defaults, unknown keys ignored
Storage utility	missing key, corrupt JSON (`{not json`, `undefined`, `[1,2`), storage that throws (blocked), full quota	fallback value, `false` from writes, nothing ever throws; the cart store starts empty from corrupted data and keeps working in memory

3. Integration Tests: jsdom + Mock Service Worker

The real modules are rendered into the real page markup (`catalog.html`, `cart.html`, loaded by `tests/support/page.js`), with the real fetch client, Axios module, URL state and localStorage stores. Only the network is replaced, by MSW. Assertions read what the shopper sees, through roles, accessible names and text (`tests/support/screen.js`, a small Testing-Library-style query set).

Module	Answer	What the test checks (as the user sees it)
Catalog	200	6 product links, "Showing 1–6 of 12 products", a price, no alert
Catalog	200 + filter	clicking the "Electronics (4)" radio sends `category=electronics`, shows exactly the 4 electronics, writes the URL
Catalog	empty	heading "No products match your filters", "Clear all filters" button, no links
Catalog	401	alert with "Please sign in again", "Error 401", "Try again"
Catalog	429	"Too many requests", "Wait 30 seconds" (from Retry-After), disabled "Try again in 30 s"
Catalog	500	"Something went wrong on our side"; after the API recovers, "Try again" loads the products
Cart	200	stored lines re-checked; "Wireless Headphones Pro is now $199.99 (was $189.99)."; totals Items 3, Subtotal/Total $263.99, Shipping Free
Cart	200 + remove	removing a line updates Subtotal $32.00, Shipping $7.99, Total $39.99 and "Add $68.00 more for free shipping."
Cart	empty	"Your cart is empty.", "Browse products" link, no "Clear cart", Total $0.00, **no API request**
Cart	404	a product no longer sold stays visible with the out-of-stock note and is not charged
Cart	401 / 429 / 500	the stored cart stays, and the shopper is told the prices could not be checked

4. End-to-End Tests (Playwright Test)

Both run in Chromium against the **official TechSphere API** (`api/techsphere/server.py`) and the site served with `VITE_API_BASE`, exactly as in grading. `playwright.config.mjs` starts both servers; each test resets the API data first.

Test	Steps and checks
filter the catalog by category and add a product to the cart	"Showing 1–8 of 24 products" → check "Audio" → URL has `category=audio`, "Showing 1–6 of 6", 6 links all "Audio" → open "Orbit Audio 3" → "Add to cart" → "Added 1 × Orbit Audio 3 to your cart." and the menu link "Cart, 1 item" → "View cart" → the line and Total "EGP 2,499.00", still there after a reload
sign in (wrong password first) and view the orders	account page while signed out → login page with "Please sign in to see your account." → wrong password → alert "The email or password is incorrect…", still on the login page, still "Sign in" → right password → account page "Signed in as Test Student (…)", orders o-001 … o-005 with date and EGP total → "Next" → o-006 … o-009

They fail when the feature breaks (`npm run e2e:prove`; full output in `docs/PR_TASK5.md`):

Deliberate bug	Test that caught it	Failing assertion
category filter not sent to the API (`utils/query-string.js`)	catalog/cart	`getByText('Showing 1–6 of 6 products')` not visible
"Add to cart" does not change the cart (`state/cart.js`)	catalog/cart	link "Cart, 1 item" not visible
vague message for a wrong password (`components/auth-form.js`)	sign-in/orders	alert "The email or password is incorrect" not visible
account requests without the Bearer token (`api/http.js`)	sign-in/orders	"Signed in as …" not visible: the API answers 401 and the app sends the shopper back to sign-in

5. Coverage (Reported Honestly)

CI runs `npm run test:coverage` (Vitest, V8 provider) on `src/state/**` and `src/utils/**`. The run fails if either folder is below 80% of lines, and the job summary shows the table from `scripts/coverage-summary.mjs`.

Measured in this environment with Node's own coverage (`npm run coverage:node`). This counts the unit and API integration tests only, not the jsdom tests, because those need jsdom:

Folder	Lines covered	%
`src/state/`	748 / 749	**99.87%**
`src/utils/`	796 / 836	**95.22%**

The lowest file is `utils/dom.js` at 56.76%: it needs a DOM, so the Node run cannot reach it, and the jsdom tests in `tests/dom/dom-utils.test.js` cover it in CI. Every other file is at or above 93%. Vitest's V8 counter treats comment and blank lines differently from Node's, so the CI number will not be identical. The 80% gate applies to whichever number CI produces.

6. Continuous Integration (`.github/workflows/ci.yml`)

Triggered on every pull request and every push to `main`; each job starts from a clean checkout and `npm install`:

Job	Runs
Lint, format and build	branch-name check, `npm run lint`, `format:check`, `build`, `contract`

Job	Runs
Unit and integration tests	`npm run test:coverage` (= `npm test` + coverage thresholds), coverage table in the job summary, coverage report uploaded
End-to-end tests	Python 3.12 for the TechSphere API, `npx playwright install --with-deps chromium`, `npm run e2e`, then the older journey suite (`npm run e2e:journeys`); report and traces uploaded on failure

7. Bugs the New Tests Found (Fixed)

Found by	Bug	Fix
unit: invalid quantities	`validateQuantity(null)` was treated as 0 ("smallest quantity is 1") instead of "not a number"	`null` is invalid input (`utils/cart-math.js`)
integration: catalog 401	a 401 showed "Could not reach the store / check your internet connection", which is wrong	own message "Please sign in again" (`api/errors.js`)
integration: cart 401/429/500	when the price check failed, the cart said nothing: stale prices looked current	the cart now says the prices could not be checked (`components/cart-view.js`)
e2e against TechSphere	category radios showed raw slugs ("smart-home")	labels for the TechSphere categories; unknown slugs made readable (`utils/format.js`)

8. How This Was Verified Here, and the Limits

The workspace used to build this task has **no network access for packages**, so Vitest, jsdom, MSW, `@vitest/coverage-v8` and `@playwright/test` could not be installed. Every test file is written for the real tools, and CI runs it with them. To check the tests here, small stand-ins were used. Each one is committed and documented:

Real tool (CI)	Used here	Result here
Vitest (Node)	Node's test runner + `tests/support/vitest-shim.mjs` (`npm run test:node`)	**215 of 215** (180 unit, 35 integration)
Vitest + jsdom + MSW	the same files in **real Chromium**, with small Vitest and MSW stand-ins that patch `fetch` and `XMLHttpRequest` (`npm run test:browser`)	**17 of 17**; a network request that misses a handler fails the run
`@playwright/test`	the same spec files on the Playwright library with a stand-in for `test`/`expect` (`npm run e2e:local`)	**2 of 2**; `E2E_RUNNER=local npm run e2e:prove`: **4 of 4** bugs caught
`@vitest/coverage-v8`	Node's coverage (`npm run coverage:node`)	§5
Older suite	`npm run e2e:journeys`	**71 of 71**

Limit: the first run with the real Vitest, jsdom, MSW and `@playwright/test` happens in CI on this pull request. Differences between jsdom and Chromium, or between the stand-ins and the real libraries, would show up there first. The tests avoid APIs that jsdom lacks (layout, `innerText`, `matchMedia`) to reduce that risk.

9. Other Changes

- `npm run e2e` is now Playwright Test (the two specs); the earlier 71-run journey suite is `npm run e2e:journeys`.

- New scripts: `test:unit`, `test:integration`, `test:coverage`, `test:browser`, `coverage:node`, `e2e:local`, `e2e:prove`.
- `devDependencies`: `@playwright/test` and `@vitest/coverage-v8` added (Vitest, jsdom, MSW and Playwright were already listed).

10. Revision After the First Grading

Feedback	Cause	Change
End-to-end tests did not run in grading	`playwright.config.mjs` always started the API with Python; the grading machine has Node.js only and already runs the TechSphere API, giving its address in `VITE_API_BASE`	With `VITE_API_BASE` set, the config uses that API and starts only the site; `tests/e2e/specs/env.js` resets the same API. Without it, nothing changes (Python starts the kit's server). Checked against the grader's API with no Python on the path: **2 of 2**
Tests should fail when a protected feature is broken	Three one-character bugs went unnoticed: a free (price 0) cart line dropped, a valid stored session read as signed out, a store reloading on another key's change in another tab	`tests/unit/state-guards.test.js` (6 tests) pins down each rule; each of those bugs now fails the unit tests
Tests should query by role, label or text	The older journey suite found most elements by CSS id or class	The catalog, cart and sign-in journeys now use roles, labels and visible text (`getByRole`, `getByLabel`, `getByText`); **71 of 71** runs still pass
Security note had no "token storage" heading	The section was titled "Where the token is kept"	Retitled "Token storage: where the token is kept, and the risk we accept" (`docs/SECURITY.md`)
Pull request: merged without a GitHub pull request	The branches were merged locally (`git merge`); there is no GitHub repository for this project yet	Not changed here: it needs the repository on GitHub, a pull request opened from `feature/automated-tests-ci` with `docs/PR_TASK5.md` as its description, CI green, then merged on GitHub